

1003 L298N, Lab 1: I2C Link, Uno to ATmega328P

STEM Starter Kit Series, Part 4, Electronics Lab, Experiment 1 (standalone test draft)

Prepared for Jesse Carpenter, Carpenter Software · August 29, 2026 · Article 1003 text is not modified by this document.

Overview

Lab-1 is a complete, self-contained lab: by the end of it, the Arduino Uno and a bare ATmega328P are exchanging real I2C messages, with no motor and no joystick hardware involved. It has four parts. Part 1 builds the ATmega328P's minimum system on the breadboard, the handful of passive parts every bare AVR chip needs before it can run at all. Part 2 turns the Uno into a temporary ISP programmer and uses it to load firmware directly onto the bare chip, since that chip has no USB port of its own. Part 3 wires the I2C bus between the two boards. Part 4 verifies the link, including a deliberate fault test, so the lab proves the link works rather than just assuming it.

The firmware is adapted from your own MageMCU repository, Serial-Communication, under Serial Communication/I2C/Uno-Uno. That project's BusI2C class, message-framing pattern, and I2C addresses (0x14 for master, 0x16 for slave) carry over unchanged. What's different here is the message itself: the original Uno-Uno demo sends live joystick position data, but this lab's joystick is not wired in until a later experiment. Lab-1 instead sends a small counter that the slave must correctly receive and answer, which proves the same underlying mechanics (transmit, receive, request, respond) without needing hardware this lab doesn't have yet. The full joystick message returns, unchanged in spirit from the original, in the experiment where the joystick is actually wired.

This document, and the code and repo layout described in it, is a first draft meant for your own bench testing. Values, wiring, and code are marked where they are standard defaults rather than something confirmed against your other sources, so discrepancies you find on the bench can be corrected here before Article 1003 is drafted.

Bill of Materials

Part	Qty	Spec	Source
Arduino Uno	1	Rev3	Electronics Lab item 1
ATmega328P	1	DIP-28, bare chip, no bootloader required	Electronics Lab item 2
Crystal	1	16 MHz, HC-49 case	Matches the Uno's own clock speed
Ceramic capacitor	2	22 pF	Crystal load capacitors
Ceramic capacitor	1	0.1 uF (100 nF)	VCC/GND decoupling, place close to the chip
Resistor	1	10 k Ohm, 1/4 W	Reset pull-up
Tactile push button	1	Momentary, normally open	Manual reset
Resistor	2	10 k Ohm, 1/4 W	I2C pull-ups, SDA and SCL (matches the 10K pull-ups in your own MCU I2C reference diagram)

Part	Qty	Spec	Source
Electrolytic capacitor	1	10 uF	Temporary, across the Uno's own RESET and GND during ISP programming only (Part 2); removed afterward
Breadboard, jumper wires	-	-	Electronics Lab bench supplies
USB cable	1	Uno to computer	-

The crystal is specified at 16 MHz to match the Uno's own clock, so both chips run PlatformIO's standard `board = uno` timing and fuse assumptions without adjustment. The pull-up and decoupling values are standard defaults; the 10 k Ohm I2C pull-up value specifically matches the number visible in your own MCU I2C.pdf reference diagram in the Uno-Uno repository.

Part 1: Build the ATmega328P Minimum System

This is the smallest working circuit a bare ATmega328P needs: power, a clock, and a reset line. Nothing here is I2C-specific yet; it is the same starting point any bare-chip AVR project would use.

- 1 Place the ATmega328P in the breadboard, straddling the center gap, oriented so pin 1 (the notch or dot end) is at a position you'll remember.
- 2 Wire pin 7 (VCC) and pin 20 (AVCC) to the breadboard's +5V rail. Wire pin 8 (GND) and pin 22 (GND) to the ground rail.
- 3 Place the 0.1uF decoupling capacitor directly across the VCC and GND rails, as close to the chip as the breadboard allows.
- 4 Wire the 16 MHz crystal between pin 9 (XTAL1) and pin 10 (XTAL2).
- 5 Add one 22pF capacitor from each crystal leg to the ground rail (two capacitors total, one per leg).
- 6 Wire pin 1 (RESET) to +5V through the 10 k Ohm pull-up resistor.
- 7 Wire the tactile button between pin 1 (RESET) and ground, so pressing it pulls reset low.
- 8 Do not wire anything else yet. Before moving to Part 2, check with a multimeter, per the measurement-first approach from Article 1000, that VCC reads +5V and that pin 1 reads +5V at rest (button not pressed) and 0V while the button is held.

Part 2: Program the Bare Chip via Arduino as ISP

The bare chip has no USB port, so it cannot be programmed the normal Arduino way. This part temporarily repurposes the Uno itself as an ISP (in-system) programmer, then uses it to upload the Lab-1 slave firmware directly onto the bare chip, with no bootloader involved at all. Because the chip never needs a bootloader for this lab (it stays wired into the breadboard, not connected to a computer), skipping the bootloader is simpler than installing one.

- 1 In the Arduino IDE, open File > Examples > 11.ArduinoISP > ArduinoISP, and upload that sketch to the Uno over its normal USB connection. The Uno is now, temporarily, an ISP programmer.
- 2 Add the 10uF electrolytic capacitor between the Uno's own RESET pin and GND, negative leg to GND. This prevents the Uno from resetting itself in the middle of programming the target chip. This capacitor is temporary and comes out again at the end of this part.
- 3 Wire the Uno to the bare chip: Uno pin 10 to chip pin 1 (RESET), Uno pin 11 (MOSI) to chip pin 17 (MOSI), Uno pin 12 (MISO) to chip pin 18 (MISO), Uno pin 13 (SCK) to chip pin 19 (SCK), plus a shared 5V and GND between the two boards.
- 4 In the IDE, set Tools > Programmer to "Arduino as ISP" and Tools > Board to "Arduino Uno" (the board setting describes the target chip's fuse and clock configuration, not the programmer).
- 5 Build the Lab-1 slave firmware (source in Part 6 below) for the `slave` PlatformIO environment and upload it with "Upload Using Programmer" rather than the normal Upload button. This writes the compiled program straight to the chip over the ISP pins, no bootloader involved.
- 6 Remove the 10uF capacitor from the Uno's RESET-GND, and remove the four ISP wires (pins 10, 11, 12, 13) between the Uno and the bare chip. The bare chip now holds the slave firmware permanently, until it is reprogrammed the same way again.

If you would rather the bare chip carry a standard Arduino bootloader instead, so it can later be reprogrammed over a USB-to-serial adapter without borrowing the Uno again, use "Burn Bootloader" before the upload step above, then upload normally afterward. That path needs a USB-to-serial adapter you don't yet have on the Bill of Materials; the direct, no-bootloader path above needs nothing beyond the Uno.

Part 3: Wire the I2C Link

With the slave firmware already on the bare chip, the Uno is now free to go back to being the I2C master.

- 1 Upload the Lab-1 master firmware (source in Part 6 below) to the Uno, over its normal USB connection, using the `master` PlatformIO environment.
- 2 Wire SDA: Uno pin A4 to chip pin 27 (PC4/SDA).
- 3 Wire SCL: Uno pin A5 to chip pin 28 (PC5/SCL).
- 4 Add a 10 k Ohm pull-up resistor from the SDA line to +5V, and a second 10 k Ohm pull-up from the SCL line to +5V.
- 5 Tie the Uno's GND and the bare chip's GND together, if they are not already sharing a ground rail from Part 1.
- 6 Power the bare chip's VCC from the Uno's 5V pin (or a bench supply sharing the same ground), so both chips are live at the same time.

Part 4: Verify

- 1 Open the Serial Monitor on the Uno at 9600 baud.
- 2 Confirm the master prints a successful transmission message and a matching "OK" result on every cycle, roughly twice a second.
- 3 Press the tactile reset button on the bare chip. Confirm the slave resumes answering correctly afterward, proving the reset circuit built in Part 1 actually works.
- 4 Disconnect the SDA or SCL jumper briefly. Confirm the master's Serial output now reports an I2C error (from BusI2C's error codes, for example a NACK or timeout) instead of a silent freeze.
- 5 Reconnect the wire and confirm the link recovers on its own within a few cycles, without needing either board to be reset.

Passing all four checks means Lab-1 is complete: the bare chip is alive, its reset circuit works, the I2C link carries real two-way traffic, and the link both fails visibly and recovers when disturbed, rather than only working by coincidence.

Part 5: Companion Repository

Proposed name: **1003-L298N-Lab1-I2C-Uno-ATmega328P**. It follows your existing repo style (hyphenated, descriptive) while carrying the article and lab number so it's easy to place alongside the other nine experiment repos this article will eventually need. If you'd rather drop the article number and name it purely by hardware, matching Joystick-Uno-L298N's style more closely, that works too; rename freely, this is only a starting suggestion.

A ready-to-use seed for this repository is attached alongside this PDF as a zip file. Its layout mirrors the Uno-Uno repository's own convention:

```
1003-L298N-Lab1-I2C-Uno-ATmega328P/  
  README.md  
  platformio.ini          (env:master and env:slave)  
  include/  
    BusI2C.h              (vendored unchanged from Serial-Communication)  
    Common.h              (trimmed to Lab-1's addresses and message sizes)  
  src/  
    MasterUno/main.cpp     (runs on the Uno)  
    SlaveMCU/main.cpp     (runs on the bare ATmega328P)
```

BusI2C.h is vendored unchanged (only the header comment changed) so this repository has no dependency on the rest of Serial-Communication. Common.h is trimmed down: the joystick, button, and LED defines from the original Uno-Uno Common.h are left out here, since Lab-1 doesn't use that hardware; they come back, unchanged in spirit, in the experiment that ports the real joystick message.

Part 6: Firmware Walkthrough

Common.h

Defines the two I2C addresses carried over from Uno-Uno (0x14 for the master, 0x16 for the slave), the size of each direction's message, and the two byte values this lab's protocol uses: a ping command and an OK status.

include/Common.h

```
#define MASTER_ADDR_0x14 0x14
#define SLAVE_ADDR_0x16 0x16

#define I2C_TX_MESSAGE_SIZE 2 // [command byte, counter byte]
#define I2C_RX_MESSAGE_SIZE 2 // [status byte, echoed counter + 1]

#define CMD_PING 0x01
#define STATUS_OK 0x01
```

BusI2C.h

Vendored unchanged from your Code Utilities folder. Its four methods matter for Lab-1: **Begin** wraps `Wire.begin` with a bus timeout, so a stuck bus recovers instead of hanging forever; **TransmitMessage** and **RequestMessage** are the master's send and fetch-the-reply calls; **ReceiveMessage** is what the slave calls inside its `onReceive` callback; **ErrorMsg** and **GetErrorNumberI2C** report the TWI error codes the verification steps in Part 4 rely on. The full class listing is in the attached repo seed; it is not reproduced here since nothing in it changed for this lab.

MasterUno/main.cpp

Runs on the Uno. Every 500 milliseconds, it sends a two-byte message (a ping command and the current counter) to the slave, then immediately requests the slave's reply and checks that the reply equals the counter it sent plus one. Every result, success or I2C error, prints to the Serial Monitor.

src/MasterUno/main.cpp

```

#include <Arduino.h>
#include <Wire.h>

#include "Common.h"
#include "BusI2C.h"

dsg::BusI2C busI2C;
uint8_t pingCounter = 0;

void setup()
{
    Serial.begin(9600);
    while (!Serial) { }

    busI2C = dsg::BusI2C();
    busI2C.Begin(MASTER_ADDR_0x14, 3000, true);

    Serial.println("Experiment 1: I2C Master ready.");
}

void pingSlave()
{
    uint8_t txMessage[I2C_TX_MESSAGE_SIZE];
    txMessage[0] = (uint8_t)CMD_PING;
    txMessage[1] = pingCounter;

    busI2C.TransmitMessage(SLAVE_ADDR_0x16, txMessage, I2C_TX_MESSAGE_SIZE);

    Serial.print("TX counter=");
    Serial.print(pingCounter);
    Serial.print(" ");
    Serial.println(busI2C.ErrorMsg());

    if (busI2C.GetErrorNumberI2C() == 0)
    {
        uint8_t rxMessage[I2C_RX_MESSAGE_SIZE];
        busI2C.RequestMessage(SLAVE_ADDR_0x16, rxMessage, I2C_RX_MESSAGE_SIZE);

        bool ok = (rxMessage[1] == (uint8_t)(pingCounter + 1));

        Serial.print("RX status=");
        Serial.print(rxMessage[0]);
        Serial.print(" echoedPlusOne=");
        Serial.print(rxMessage[1]);
        Serial.println(ok ? " OK" : " MISMATCH");
    }

    pingCounter++;
}

void loop()
{
    static uint32_t lastSend = 0;
    if (millis() - lastSend >= 500)
    {
        lastSend = millis();
        pingSlave();
    }

    busI2C.ClearTimeout();
}

```

SlaveMCU/main.cpp

Runs on the bare ATmega328P. It has no Serial output since that chip has no USB connection. Two callbacks do the work: **onReceiveI2C** stores whatever the master just sent, and **onRequestI2C** answers with that stored counter plus one, which is what proves the slave actually processed the message rather than only acknowledging its address on the bus. One difference from the original SlaveUno code: the callbacks are registered once in `setup()`, not reassigned repeatedly inside `loop()`.

src/SlaveMCU/main.cpp

```
#include <Arduino.h>
#include <Wire.h>

#include "Common.h"
#include "BusI2C.h"

dsg::BusI2C busI2C;

volatile uint8_t lastCommand = 0;
volatile uint8_t lastCounter = 0;

void onReceiveI2C(int numberBytes)
{
    uint8_t rxMessage[I2C_TX_MESSAGE_SIZE];
    busI2C.ReceiveMessage(rxMessage, numberBytes);

    lastCommand = rxMessage[0];
    lastCounter = rxMessage[1];
}

void onRequestI2C()
{
    uint8_t txMessage[I2C_RX_MESSAGE_SIZE];
    txMessage[0] = (uint8_t)STATUS_OK;
    txMessage[1] = (uint8_t)(lastCounter + 1);

    Wire.write(txMessage, I2C_RX_MESSAGE_SIZE);
}

void setup()
{
    busI2C = dsg::BusI2C();
    busI2C.Begin(SLAVE_ADDR_0x16, 3000, true);

    Wire.onReceive(onReceiveI2C);
    Wire.onRequest(onRequestI2C);
}

void loop()
{
    busI2C.ClearTimeout();
}
```

platformio.ini

One project, two environments, the same pattern Uno-Uno used to switch which folder compiles. The `slave` environment overrides the upload protocol to use the Uno-as-ISP path from Part 2 instead of a normal serial upload.

platformio.ini

```
[env:master]
platform = atmelavr
board = uno
framework = arduino
build_src_filter = +<*> -<SlaveMCU/>
monitor_speed = 9600
upload_port = /dev/ttyACM0
```

```
[env:slave]
platform = atmelavr
board = uno
framework = arduino
build_src_filter = +<*> -<MasterUno/>
upload_protocol = stk500v1
upload_speed = 19200
upload_port = /dev/ttyACM0
upload_flags =
  -P$UPLOAD_PORT
  -b$UPLOAD_SPEED
```

Troubleshooting

Symptom	Likely cause
Master's Serial Monitor shows nothing at all	Wrong baud rate selected, or the Uno reset during upload and the sketch hasn't restarted yet.
Every TX reports a NACK-on-address error	Slave chip not powered, SDA/SCL swapped, or the slave firmware wasn't actually written during Part 2 (wrong environment uploaded).
TX succeeds but RX times out or returns garbage	Missing common ground between the two boards, or a missing/wrong-value pull-up resistor.
RX status is OK but echoedPlusOne never matches	Off-by-one or byte-order bug; check that Common.h's message sizes match on both boards, and that both were rebuilt after any edit.
ISP upload in Part 2 fails immediately	ArduinoISP sketch not actually running on the Uno, the 10uF capacitor missing on the Uno's RESET, or one of the four ISP wires (10/11/12/13) miswired.
Bare chip never seems to start (no reaction to reset button)	Crystal or load capacitors miswired, or VCC/GND not reaching both pin pairs (7/20 and 8/22).

This is a first draft, generated from your own MageMCU Uno-Uno code and standard Arduino/AVR practice, not yet verified on your bench. Everything above, component values, pin assignments, the ISP method, and the code itself, is open to correction once you've tested it against your other sources and the physical hardware.